

Diseño y Análisis de Algoritmos

Taller 1: Invariante de Loop

Prof. Felipe Ruiz, Ayud. Ferran Rojas Andreu

Entrega del informe: [FECHA ENTREGA]

Presentación oral: [FECHA PRESENTACIÓN]

Modalidad: En parejas

Introducción

El **invariante de loop** es una herramienta fundamental para probar la correctitud de algoritmos iterativos. Un invariante es una propiedad que se mantiene verdadera antes y después de cada iteración de un bucle. Para demostrar que un algoritmo es correcto, debemos probar tres propiedades:

1. **Inicialización:** El invariante es verdadero antes de la primera iteración.
2. **Mantenimiento:** Si el invariante es verdadero antes de una iteración, sigue siéndolo antes de la siguiente.
3. **Terminación:** Cuando el loop termina, el invariante (junto con la condición de salida) implica la postcondición deseada.

Este taller tiene como objetivo que apliquen esta técnica a un algoritmo clásico, desarrollando una prueba formal de correctitud y comunicándola efectivamente.

Algoritmos Sugeridos

Cada pareja debe elegir **uno** de los siguientes algoritmos. Deben analizar el algoritmo, definir su invariante de loop, probar las tres propiedades y analizar su complejidad.

1. Insertion Sort (CLRS Cap. 2.1)

Ordena un arreglo insertando cada elemento en su posición correcta dentro de la porción ya ordenada.

Pseudocódigo

```
1  procedure INSERTION-SORT( $A$ )
2    for  $j = 2$  to  $A.length$ 
3       $key = A[j]$                                 ▷ Insert  $A[j]$  into sorted sequence  $A[1..j-1]$ 
4       $i = j - 1$ 
5      while  $i > 0$  and  $A[i] > key$ 
6         $A[i+1] = A[i]$ 
7         $i = i - 1$ 
8       $A[i+1] = key$ 
```

Invariante Sugerido

Al inicio de cada iteración del loop exterior (índice j):

$A[1..j - 1]$ contiene los elementos originales de $A[1..j - 1]$ en orden no decreciente.

2. Kadane's Algorithm — Máximo Subarreglo (CLRS Cap. 4.1)

Encuentra el subarreglo contiguo de suma máxima en tiempo lineal.

Pseudocódigo

```

1  procedure KADANE( $A$ )
2       $max\_ending\_here = A[1]$ 
3       $max\_so\_far = A[1]$ 
4      for  $i = 2$  to  $A.length$ 
5           $max\_ending\_here = \max(A[i], max\_ending\_here + A[i])$ 
6           $max\_so\_far = \max(max\_so\_far, max\_ending\_here)$ 
7      return  $max\_so\_far$ 
```

Invariante Sugerido

Al inicio de la iteración i (después de procesar $A[1..i - 1]$):

- $max_ending_here = \max\{\sum_{k=t}^{i-1} A[k] \mid 1 \leq t \leq i - 1\}$
- $max_so_far = \max\{\sum_{k=l}^r A[k] \mid 1 \leq l \leq r \leq i - 1\}$

3. Sliding Window — Suma Máxima de Ventana Fija k

Dado un arreglo A y un entero k , encuentra la suma máxima de cualquier subarreglo contiguo de longitud exactamente k .

Pseudocódigo

```

1  procedure MAX-SLIDING-WINDOW-SUM( $A, k$ )                                ▷ Requiere  $1 \leq k \leq A.length$ 
2       $window\_sum = \sum_{i=1}^k A[i]$ 
3       $best = window\_sum$ 
4      for  $i = k + 1$  to  $A.length$ 
5           $window\_sum = window\_sum + A[i] - A[i - k]$ 
6           $best = \max(best, window\_sum)$ 
7      return  $best$ 
```

Invariante Sugerido

Al inicio de la iteración i (para $i > k$):

- $window_sum = \sum_{t=i-k}^{i-1} A[t]$
- $best = \max\{\sum_{t=j-k+1}^j A[t] \mid k \leq j \leq i - 1\}$

4. Binary Search (Búsqueda Binaria, CLRS Cap. 2.3)

Busca un elemento **target** en un arreglo ordenado A reduciendo el espacio de búsqueda a la mitad en cada paso.

Pseudocódigo

```
1  procedure BINARY-SEARCH( $A, target$ )
2       $lo = 1$ 
3       $hi = A.length$ 
4      while  $lo \leq hi$ 
5           $mid = \lfloor (lo + hi)/2 \rfloor$ 
6          if  $A[mid] = target$ 
7              return  $mid$ 
8          else if  $A[mid] < target$ 
9               $lo = mid + 1$ 
10         else
11              $hi = mid - 1$ 
12     return NIL
```

Invariante Sugerido

Al inicio de cada iteración del **While**:

Si $target$ existe en $A[1..n]$, entonces $target \in A[lo..hi]$.

Preguntas Guía para el Análisis

Aplicar las siguientes preguntas al algoritmo elegido:

1. **Definición formal:** Escriba el invariante $I(i)$ usando notación matemática precisa (cuantificadores, sumatorios, índices).
2. **Inicialización:** Pruebe que $I(i_0)$ es verdadero antes de la primera iteración ($i_0 = 1$ o 2 según el loop).
3. **Mantenimiento:** Asumiendo $I(i)$ verdadero al inicio de la iteración i , demuestre que $I(i + 1)$ es verdadero al inicio de la siguiente. Sea explícito en cómo cada línea del pseudocódigo preserva el invariante.
4. **Terminación:** Identifique la condición de salida del loop. Demuestre que $I(i_{final})$ junto con la negación de la condición de loop implica la postcondición del algoritmo.
5. **Complejidad:** Analice complejidad temporal (mejor, peor, promedio) y espacial. Justifique cada término.
6. **Edge cases:** Discuta comportamiento con: arreglo vacío, un elemento, duplicados, ya ordenado, todos negativos, $k = n$, target fuera de rango, etc.
7. **Variantes del invariante:** ¿Existe una versión más fuerte del invariante que también se mantenga? ¿Una más débil que no sea suficiente para probar correctitud? Explique.
8. **Trazas para presentación:** Prepare 2–3 ejemplos concretos con traza paso a paso (valores de variables en cada iteración) para usar en su demo en vivo.

Entregables

1. Informe Escrito (2–3 páginas, PDF)

- Pseudocódigo estilo CLRS del algoritmo elegido.
- Definición formal del invariante $I(i)$.
- Pruebas de Inicialización, Mantenimiento, Terminación.
- Análisis de complejidad con justificación.
- Discusión de edge cases y variantes del invariante.
- Formato: márgenes 1in, fuente 11pt, interlineado 1.15.

2. Presentación Oral (5–7 minutos, en parejas)

- 8–10 diapositivas (poco texto, diagramas, trazas).
- Demo en vivo: recorrer un ejemplo paso a paso mostrando cómo el invariante se mantiene.
- Explicación clara e intuitiva del invariante (no solo leer la prueba formal).
- Ambos integrantes participan; tiempo equilibrado.
- Preguntas del ayudante al final.

Formato del Pseudocódigo (Estilo CLRS)

Usen el paquete `algpseudocode` como en los ejemplos arriba. Puntos clave:

- Estructuras: `\Procedure`, `\For`, `\While`, `\If`, `\State`.
- Comentarios: `\Comentario{...}` alineados a la derecha.
- Variables en math mode: `$A$`, `$key$`, `$i$`.
- Índices 1-based (estilo CLRS) o 0-based (consistente en todo el documento).
- Palabras clave en **negrita**: **to**, **and**, **or**, **not**.

Rúbrica de Evaluación (100 pts)

Categoría	Pts	Criterios
Correctness	30	Invariante $I(i)$ bien definido; Inicialización/Mantenimiento/Terminación probados formalmente; Complejidad temporal/espacial correcta con justificación; Edge cases cubiertos
Clarity & Structure	25	Pseudocódigo estilo CLRS limpio y consistente; Organización lógica (def $\rightarrow$ invariante $\rightarrow$ prueba $\rightarrow$ complejidad); Notación matemática precisa; Ejemplos de traza ilustrativos
Depth of Analysis	25	Identifica variantes del invariante (más fuerte/débil); Discute pre/postcondiciones; Compara con alternativas; Generaliza a familia de algoritmos; Menciona limitaciones
Presentation Quality	20	Slides visuales (≤ 10); Demo en vivo paso a paso; Responde preguntas con dominio; Tiempo 5–7 min; Trabajo en parejas equilibrado

Total: 100 pts

¡Éxito en su análisis!